Nexus / Artifact Repository ⭐ ⭐ — Interview-Ready Q&A

1. What is Nexus Repository?
Nexus Repository is an **artifact repository manager** used to store, manage, and distribute software artifacts created during the build process.
Examples of artifacts are JAR files, WAR files, Docker images, npm packages, Python packages, etc.
Simple ga gurthupettuko:
Nexus = **Build artifacts store + manage chese place.**
Eng: Nexus Repository is an artifact management tool used to store, version, and distribute build artifacts and packages.

---

2. Why do we need an Artifact Repository?
Build pipeline application ni compile/build chesina tarvata output generate avtundi.
For example, Java application ki .jar file generate avtundi. Aa artifact ni directly server ki send cheyyakunda repository lo store chesi, deployment pipeline tarvata required version ni download chesukuntundi.
Simple ga gurthupettuko:
Build → Artifact → Nexus → Deployment
Eng: An artifact repository provides a centralized and versioned location to store build outputs so they can be reused and deployed consistently across environments.

---

3. What is an Artifact?
An artifact is a **build output/package** produced by the application build process.
Examples:
Java → .jar / .war
Node.js → npm package
Python → Python package
Docker → Container image
Simple ga gurthupettuko:
Artifact = **Build ayyaka vachina deployable package.**
Eng: An artifact is a versioned build output or package that can be stored and later consumed by deployment or other build processes.

---

4. What is an Artifact Repository?
An Artifact Repository is a centralized system where application packages and build artifacts are stored.
It provides versioning, access control, retention, and distribution capabilities.
Examples include Nexus Repository, JFrog Artifactory, and cloud-native artifact services.
Simple ga gurthupettuko:
Artifact Repository = **Software packages ki storage.**
Eng: An artifact repository centrally stores and distributes versioned software packages and build outputs.

---

5. What is the difference between Git and Nexus?
Git stores **source code**.
Nexus stores **build artifacts/packages**.
Example:
Developer
↓
Git → Source Code
↓
Jenkins Build
↓
Nexus → JAR / WAR / Package
↓
Deployment
Simple ga gurthupettuko:
Git = Source Code
Nexus = Build Artifacts
Eng: Git is primarily used for source-code version control, while Nexus is used to store and distribute versioned build artifacts and packages.

---

6. What is a Repository in Nexus?
A Repository is a logical storage location for a particular package format or artifact type.
For example, we can have separate repositories for Maven artifacts and Docker images.
Simple ga gurthupettuko:
Repository = **Artifacts store cheyyadaniki logical location.**
Eng: A Nexus Repository is a logical location used to store and manage artifacts of a particular package format or purpose.

7. What is a Maven Repository?
A Maven Repository stores Java/Maven artifacts such as:
.jar
.war
.pom
For example, after Jenkins builds a Java application, the generated JAR can be uploaded to a Maven repository in Nexus.
Simple ga gurthupettuko:
Maven Repository = **Java JAR/WAR artifacts store.**
Eng: A Maven repository in Nexus stores and distributes Maven-compatible Java artifacts such as JARs, WARs, and POM files.

---

8. What is a Docker Repository in Nexus?
A Docker Repository is used to store Docker container images.
For example:
myapp:1.0
myapp:1.1
myapp:2.0
CI pipeline Docker image build chesi Nexus Docker repository ki push cheyyachu.
Simple ga gurthupettuko:
Docker Repository = **Docker Images store.**
Eng: A Docker repository in Nexus stores and distributes versioned container images for use by deployment systems.

---

9. What is the difference between Docker Registry and Nexus?
A Docker Registry is designed to store and distribute Docker/OCI container images.
Nexus is a broader artifact repository manager that can manage multiple package formats, including Maven and Docker, depending on the repository configuration and supported formats.
Simple ga gurthupettuko:
Docker Registry = Mainly Container Images
Nexus = **Multiple Artifact Types**
Eng: A Docker registry focuses on container images, while Nexus can provide centralized artifact management for multiple package formats.

---

10. What is Artifact Versioning?
Artifact versioning means maintaining different versions of the same application artifact.
For example:
payment-service
├── 1.0
├── 1.1
├── 1.2
└── 2.0
This allows teams to deploy a specific known version and roll back when required.
Simple ga gurthupettuko:
Version = **Which build/package deploy cheyyalo identify cheyyadam.**
Eng: Artifact versioning allows teams to uniquely identify, retrieve, deploy, and roll back to specific application builds.

---

11. Why is versioning important in DevOps?
Suppose Production lo v2.0 deploy chesam and application issue vachindi.
If v1.9 is available in Nexus, deployment pipeline can retrieve the previous known-good artifact and deploy it.
Simple ga gurthupettuko:
v2.0 failed → Nexus → v1.9 → Rollback
Eng: Versioning provides traceability and makes it easier to reproduce deployments and roll back to a known-good artifact.

---

12. How does Jenkins integrate with Nexus?
Jenkins performs the application build.
After a successful build and testing, Jenkins uploads the generated artifact to Nexus.
Example:
Git
↓
Jenkins
↓
Build + Test
↓
JAR generated
↓
Nexus
↓
Artifact stored
Simple ga gurthupettuko:
Jenkins = Build
Nexus = Store Artifact
Eng: Jenkins can build and test the application and then publish the resulting versioned artifact to Nexus for later deployment.

13. How does deployment use Nexus?
The deployment pipeline identifies the required artifact version from Nexus and downloads it.
For example:
Nexus
↓
payment-service-2.0.jar
↓
Deployment Server / EC2
↓
Application Start
This separates the build process from the deployment process.
Simple ga gurthupettuko:
Build Once → Store → Deploy Same Artifact
Eng: The deployment pipeline retrieves a specific versioned artifact from Nexus and deploys that artifact to the target environment.

---

14. What is the advantage of Build Once, Deploy Many?
We build the application once and store the artifact in Nexus.
The exact same artifact can then be promoted through:
Dev → QA → UAT → Production
We don't rebuild the application separately for each environment.
This improves consistency and reduces the chance of environment-specific build differences.
Simple ga gurthupettuko:
Build Once → Dev → QA → Prod
Eng: Build Once, Deploy Many ensures the same tested artifact is promoted across environments instead of generating different builds for each environment.

---

15. What is a Snapshot version?
A Snapshot version generally represents an artifact under active development rather than a final stable release.
For example:
1.0-SNAPSHOT
It can change as development continues.
Simple ga gurthupettuko:
Snapshot = **Development / changing version.**
Eng: A snapshot version represents a development build that may change before a final release version is produced.

---

16. What is a Release version?
A Release version represents a finalized, identifiable artifact intended for stable consumption.
Example:
1.0
1.1
2.0
Once released, teams generally expect that exact version to remain reproducible.
Simple ga gurthupettuko:
Release = **Stable version.**
Eng: A release version identifies a stable artifact intended for controlled consumption and deployment.

---

17. What is a Hosted Repository?
A Hosted Repository is where your organization **stores its own artifacts**.
For example, Jenkins builds a company Java application and uploads the JAR to a hosted Maven repository.
Simple ga gurthupettuko:
Hosted = **Mana own artifacts store.**
Eng: A hosted repository stores artifacts produced and managed by the organization itself.

---

18. What is a Proxy Repository?
A Proxy Repository acts as a cache between your organization and an external artifact repository.
For example, instead of every build downloading a dependency directly from a public Maven repository, Nexus can retrieve and cache it.
Simple ga gurthupettuko:
Proxy = **External repository ni access + cache.**
Eng: A proxy repository retrieves artifacts from an external repository and can cache them locally for controlled and faster reuse.

---

19. What is a Group Repository?
A Group Repository provides a single endpoint that can combine multiple repositories.
For example:
Group Repository
├— Hosted Repository
└— Proxy Repository
Developers or build tools can use one URL instead of configuring multiple repository URLs.
Simple ga gurthupettuko:
Group = **Multiple repositories → One endpoint.**
Eng: A group repository provides a single access point that combines multiple repositories for easier artifact consumption.

20. Difference between Hosted, Proxy, and Group Repository?
Hosted → Own artifacts
Proxy → External artifacts/cache
Group → Multiple repositories through one endpoint
For example, a Maven build can use a group repository that provides access to both internal company artifacts and cached external dependencies.
Simple ga gurthupettuko:
Hosted = Own
Proxy = External
Group = Combine
Eng: Hosted repositories store internal artifacts, proxy repositories cache external artifacts, and group repositories provide unified access to multiple repositories.

---

21. How do you upload an artifact to Nexus?
Usually the CI/CD pipeline performs the upload after a successful build and test.
For a Maven project, Maven configuration can point to the Nexus repository and the pipeline can publish the generated artifact.
Typical flow:
Git → Build → Test → Package → Publish → Nexus
Simple ga gurthupettuko:
Build Success → Artifact → Nexus Upload
Eng: Artifacts are typically uploaded to Nexus automatically from the CI pipeline after successful build and validation.

---

22. How do you download an artifact from Nexus?
The deployment process specifies the required artifact version and downloads it from Nexus.
For example:
Nexus
↓
myapp-2.0.jar
↓
Deployment Server
The deployment should use a specific version rather than an unpredictable latest build.
Simple ga gurthupettuko:
Version Select → Nexus Download → Deploy
Eng: The deployment pipeline retrieves the required versioned artifact from Nexus and deploys it to the target environment.

---

23. How do you secure Nexus?
I would implement:

- Authentication and authorization
- Role-based access control
- HTTPS
- Separate permissions for developers and CI/CD
- Restricted repository access
- Secure credentials
- Artifact retention policies

CI/CD credentials should have only the permissions required for their operations.
Simple ga gurthupettuko:
Nexus Security = **Auth + RBAC + HTTPS + Least Privilege**
Eng: I secure Nexus using authentication, role-based access control, HTTPS, least-privilege permissions, and controlled access for users and CI/CD systems.

---

24. What happens if Jenkins successfully builds but cannot upload to Nexus?
First I check the Jenkins error message.
Then I verify:
Nexus URL
↓
Credentials
↓
Repository name
↓
Permissions
↓
Network connectivity
↓
Artifact configuration
I also verify whether the Nexus repository is available and whether the CI service account has permission to upload artifacts.
Simple ga gurthupettuko:
Build Success → Upload Fail → Credentials / URL / Permissions / Network
Eng: If artifact upload fails, I check Nexus connectivity, repository configuration, credentials, permissions, and the CI pipeline's publishing configuration.

25. What happens if the deployment cannot download an artifact from Nexus?
First I check whether the requested artifact and version actually exist.
Then I verify Nexus connectivity, authentication, repository permissions, URL, network access, and artifact coordinates.
I also check Nexus logs and deployment pipeline logs for the exact failure.
Simple ga gurthupettuko:
Artifact Exists? → Access? → Network? → Credentials? → Root Cause
Eng: I troubleshoot artifact download failures by verifying artifact availability, repository configuration, authentication, permissions, network connectivity, and logs.

---

26. How do you handle old artifacts in Nexus?
We should not keep unlimited artifacts forever because storage can grow significantly.
We can define retention or cleanup policies based on project requirements.
For example, we may retain recent release versions and remove old development artifacts after a defined period.
Simple ga gurthupettuko:
Old Artifacts → Retention Policy → Storage Control
Eng: I use artifact retention and cleanup policies to control repository growth while preserving the versions required for deployment and rollback.

---

27. Why shouldn't we use latest **blindly in deployments?**
Using latest makes it difficult to know exactly which artifact was deployed.
For Production deployments, we generally prefer an immutable or clearly identified version such as:
payment-service:2.4.1
This improves traceability and rollback.
Simple ga gurthupettuko:
latest ❌
Specific Version ✅
Eng: I prefer specific immutable artifact versions because they provide predictable deployments, traceability, and reliable rollback.

---

28. Explain Nexus in a real-time CI/CD pipeline.
Developer code ni Git lo push chestadu.
Jenkins code checkout chesi build and unit tests run chestundi. Build successful ayithe JAR/Docker image create chestundi and Nexus lo versioned artifact ga store chestundi.
Deployment pipeline Nexus nunchi same artifact ni retrieve chesi Dev, QA, and Production environments ki promote chestundi.

Developer→ Git→ Jenkins→ Build + Test→ Artifact→ Nexus→ Dev→ QA
Production
Simple ga gurthupettuko:
Git → Jenkins → Build → Nexus → Deploy
Eng: In a real-time CI/CD pipeline, Jenkins builds and validates the application, publishes a versioned artifact to Nexus, and the deployment pipeline retrieves the same artifact and promotes it across environments.

---

29. What is the biggest advantage of Nexus in DevOps?
The biggest advantage is **centralized and versioned artifact management**.
It separates the build stage from the deployment stage and allows teams to reliably store, retrieve, promote, and roll back application artifacts.
Simple ga gurthupettuko:
Nexus = **Central Artifact Storage + Versioning + Reuse**
Eng: Nexus provides centralized, versioned artifact management that improves consistency, traceability, and reliability across CI/CD pipelines.

---

30. Explain Build → Artifact → Deploy concept.
The application source code is first stored in Git.
CI builds and tests the source code. The output becomes an artifact, which is stored in Nexus with a specific version.
During deployment, the pipeline retrieves that exact artifact and deploys it.
Source Code
↓
Build
↓
Test
↓
Artifact
↓
Nexus
↓
Deploy Same Artifact
Simple ga gurthupettuko:
Code → Build → Artifact → Nexus → Deploy
Eng: The Build–Artifact–Deploy model separates source code, build output, and deployment, allowing the same tested and versioned artifact to be reliably promoted across environments.
ChatGPT is AI and can make mistakes.
You'll get smarter responses and can upload files, images, and more.